FXDE Plugin Runtime 完全設計書

0. 先に訂正

前回こちらが「旧 DTO がまだ残っている」と断定したのは誤りだった。あなたの確認どおり、

- `apps/api/src/modules/plugins/dto/get-plugins.query.dto.ts`
- `apps/api/src/modules/plugins/dto/plugin-id.param.dto.ts`

は存在していない前提で扱うべきやった。ここは訂正する。

現時点では **DTO 規約違反が残っていると断定せず、必要なら再確認対象** として扱うのが正しい。

---

1. 目的

本設計は、すでに実装済みの **Plugin Manager / Plugin Registry / Enable-Disable / Source Viewer** の上に、 実際に **有効化されたプラグインが FXDE のチャート・分析画面へ影響を与える Runtime 実行基盤** を追加するための詳細設計である。

この Runtime の追加により、

- Plugin を有効化しただけで終わらず
- Chart / Strategy / Signals / Prediction と連動し
- Overlay / Signal / Indicator / Analysis Result を描画・表示できる

状態を実現する。

---

2. 現状整理

現状の Plugin System は、おおむね以下まで到達している前提で整理する。

2.1 実装済み前提

- plugin_manifests
- installed_plugins
- plugin_audit_logs
- Plugin Manager UI
- Plugin Detail Drawer
- Source Preview Read Only Viewer
- Enable / Disable
- Plugin Registry（DBベース状態管理）

2.2 未実装の核心

未実装なのは **Plugin Runtime** であり、具体的には以下が不足している。

- 有効プラグインの読み込み
- 実行コンテキスト生成
- チャートデータを各プラグインへ渡す仕組み
- プラグイン結果を Overlay / Signal / Indicator として正規化する仕組み
- チャート描画系 UI への受け渡し

したがって現在は

- 管理はできる
- 有効 / 無効も切り替えられる
- しかしチャートに何も反映されない

という状態になる。

---

3. Runtime の役割

Plugin Runtime は、FXDE Core と各プラグインの間に立つ **統合実行層** である。

役割は次の 5 つ。

1. 実行対象プラグインの選別
2. 実行用データ・設定・権限の注入
3. 例外隔離と timeout 制御
4. 結果の標準フォーマット化
5. UI / Chart / Summary / Signal へ結果受け渡し

---

4. 全体アーキテクチャ

```
[Chart Page / Strategy Page]
        ↓
[Chart Query / Snapshot Query / Candles Query]
        ↓
[Plugin Runtime Coordinator]
        ↓
[Execution Context Builder]
        ↓
[Enabled Plugins Resolver]
        ↓
[Plugin Executor]
    ├── Indicator Plugins
    ├── Overlay Plugins
    ├── Signal Plugins
    └── Analysis Plugins
        ↓
```

```
[Result Normalizer]
        ↓
[Plugin Runtime Result]
        ↓
[Chart Renderer / Signal Panel / Summary Panel]
```

---

5. Runtime の適用対象

v1 の Runtime 対象は以下に限定する。

5.1 Runtime 対象

- `indicator`
- `overlay`
- `signal`
- `analysis`
- `ai`（補助要約のみ。直接描画は任意）

5.2 v1 では対象外

- 外部 connector の live execution
- 自動売買実行
- code injection
- sandbox 外部実行
- UI 上での plugin source 編集

5.3 注意

既存の PluginType 正本が `pattern / indicator / strategy / risk / overlay / signal / ai / connector` の場合は、 Runtime 対象を次のように読み替える。

- `pattern` → analysis / signal に近いものとして扱う
- `strategy` → analysis / decision aid として扱う
- `risk` → chart overlay というより panel output として扱う

つまり Runtime 設計は、**PluginType 正本を壊さずに Capability ベースで動かす** 方が安全である。

---

6. Plugin Runtime 設計思想

6.1 型だけでなく Capability で実行する

`pluginType` だけで分岐すると柔軟性が低い。したがって Runtime は

- `pluginType`
- `capabilities[]`

の両方を見る。

6.2 推奨 capability 一覧

```
export type PluginCapability =
  | 'chart.overlay'
  | 'chart.signal'
  | 'chart.indicator'
  | 'analysis.summary'
  | 'analysis.bias'
  | 'analysis.pattern-detect'
  | 'risk.score'
  | 'strategy.vote'
  | 'prediction.hint';
```

Runtime 側は capability ごとに実行ルーティングを決める。

---

7. 実行ポイント

Runtime の入口は 1 個にしない。用途別に 3 系統用意する。

7.1 Chart Runtime

目的:

- チャート描画に必要な Overlay / Signal / Indicator を返す

呼び出し元:

- Chart page
- Dashboard の mini chart
- Strategy 内 chart section

7.2 Analysis Runtime

目的:

- バイアス判定、パターン検出、スコア、サマリー材料を返す

呼び出し元:

- Strategy page
- Snapshot / Signals page

7.3 Summary Runtime

目的:

- AI summary や explanation へ渡す補助コンテキストを構築する

呼び出し元:

- AI Summary panel
- Prediction assistant

---

8. 実装スコープ

8.1 v1 で必須

- Enabled Plugins Resolver
- Plugin Execution Context Builder
- Plugin Executor
- Result Normalizer
- Chart Runtime API
- Frontend hooks for runtime result
- Chart Renderer integration

8.2 v1 でできれば実装

- 実行時間計測
- plugin error isolation
- per-plugin timeout
- dependency warning
- health check update

8.3 v1 では未実装でも可

- multi-process sandbox
- worker thread isolation
- signed plugin execution verification
- hot-reload plugin code

---

9. 標準インターフェース

9.1 Plugin Runtime Manifest 拡張

既存の `PluginManifest` に対して Runtime 用のフィールドを追加する。

```
export interface PluginRuntimeDescriptor {
  runtimeVersion: '1';
  entryExportName?: string;
  supportedHooks: PluginRuntimeHook[];
  defaultTimeoutMs?: number;
  maxLookbackBars?: number;
  outputKinds: PluginOutputKind[];
}

export type PluginRuntimeHook =
```

```
  | 'chart'
  | 'analysis'
  | 'summary';

export type PluginOutputKind =
  | 'overlay'
  | 'signal'
  | 'indicator'
  | 'analysis'
  | 'risk'
  | 'summary-context';
```

9.2 実行対象プラグインの標準 export

```
export interface FXDEPluginModule {
  manifest: PluginManifest;
  runtime?: PluginRuntimeDescriptor;
  executeChart?: FXDEChartPluginExecute;
  executeAnalysis?: FXDEAnalysisPluginExecute;
  executeSummary?: FXDESummaryPluginExecute;
}
```

9.3 Chart 実行関数

```
export type FXDEChartPluginExecute = (
  context: ChartPluginExecutionContext
) => Promise<PluginExecutionResult>;
```

9.4 Analysis 実行関数

```
export type FXDEAnalysisPluginExecute = (
  context: AnalysisPluginExecutionContext
) => Promise<PluginExecutionResult>;
```

---

10. 実行コンテキスト設計

10.1 共通コンテキスト

```
export interface BasePluginExecutionContext {
  requestId: string;
  userId: string | null;
  symbol: string;
  timeframe: string;
  timezone: string;
  locale: string;
```

```
  nowIso: string;
  pluginId: string;
  pluginSlug: string;
  settings: Record<string, unknown>;
  market:
   | {
      symbol: string;
      timeframe: string;
      candles: CandleBar[];
      latestPrice: number | null;
    };
}
```

10.2 Chart 用コンテキスト

```
export interface ChartPluginExecutionContext
  extends BasePluginExecutionContext {
  hook: 'chart';
  lookbackBars: number;
  visibleRange: {
    fromIndex: number | null;
    toIndex: number | null;
  };
  indicators?: IndicatorSnapshotMap;
  snapshot?: MarketSnapshot | null;
  trades?: UserTradeLite[];
}
```

10.3 Analysis 用コンテキスト

```
export interface AnalysisPluginExecutionContext
  extends BasePluginExecutionContext {
  hook: 'analysis';
  snapshot?: MarketSnapshot | null;
  indicators?: IndicatorSnapshotMap;
  mtf?: MultiTimeframePack | null;
  openPositions?: UserPositionLite[];
}
```

10.4 Summary 用コンテキスト

```
export interface SummaryPluginExecutionContext
  extends BasePluginExecutionContext {
  hook: 'summary';
  analysisResults?: NormalizedAnalysisOutput[];
  signalResults?: NormalizedSignalOutput[];
}
```

11. プラグイン出力の標準化

Runtime の最大の要点は、各プラグインの返却を **UI 非依存の標準データ** に正規化すること。

11.1 共通 result

```
export interface PluginExecutionResult {
  pluginId: string;
  status: 'ok' | 'error' | 'skipped' | 'timeout';
  durationMs: number;
  warnings: string[];
  outputs: PluginNormalizedOutput[];
  debug?: Record<string, unknown>;
}
```

11.2 出力 union

```
export type PluginNormalizedOutput =
  | NormalizedOverlayOutput
  | NormalizedSignalOutput
  | NormalizedIndicatorOutput
  | NormalizedAnalysisOutput
  | NormalizedRiskOutput
  | NormalizedSummaryContextOutput;
```

11.3 Overlay 出力

```
export interface NormalizedOverlayOutput {
  kind: 'overlay';
  overlayType: 'zone' | 'line' | 'band' | 'marker' | 'label';
  title: string;
  items: ChartOverlayItem[];
}
```

11.4 Signal 出力

```
export interface NormalizedSignalOutput {
  kind: 'signal';
  signalType: 'buy' | 'sell' | 'warning' | 'neutral';
  title: string;
  confidence: number | null;
  markers: ChartSignalMarker[];
  summary?: string;
}
```

11.5 Indicator 出力

```
export interface NormalizedIndicatorOutput {
  kind: 'indicator';
  indicatorKey: string;
  title: string;
  series: IndicatorSeries[];
}
```

11.6 Analysis 出力

```
export interface NormalizedAnalysisOutput {
  kind: 'analysis';
  category: 'bias' | 'pattern' | 'structure' | 'volatility' | 'momentum';
  title: string;
  score?: number | null;
  summary: string;
  details?: Record<string, unknown>;
}
```

---

12. Chart Overlay の詳細型

```
export interface ChartOverlayItemBase {
  id: string;
  color?: string;
  opacity?: number;
  label?: string;
}

export interface ChartZoneOverlayItem extends ChartOverlayItemBase {
  shape: 'zone';
  fromTime: string;
  toTime: string;
  y1: number;
  y2: number;
}

export interface ChartLineOverlayItem extends ChartOverlayItemBase {
  shape: 'line';
  fromTime: string;
  toTime: string;
  y1: number;
  y2: number;
  style?: 'solid' | 'dashed' | 'dotted';
}
```

```
export interface ChartMarkerOverlayItem extends ChartOverlayItemBase {
  shape: 'marker';
  time: string;
  price: number;
  markerType: 'arrow-up' | 'arrow-down' | 'dot' | 'triangle';
}

export type ChartOverlayItem =
  | ChartZoneOverlayItem
  | ChartLineOverlayItem
  | ChartMarkerOverlayItem;
```

---

13. Supply Demand 系プラグインの想定

例として、Supply Demand Zones PRO を runtime 対象にすると次のようになる。

13.1 plugin capabilities

```
capabilities: ['chart.overlay', 'analysis.pattern-detect']
```

13.2 executeChart 戻り値例

```
{
  pluginId: 'plg_supply_demand_pro',
  status: 'ok',
  durationMs: 12,
  warnings: [],
  outputs: [
    {
      kind: 'overlay',
      overlayType: 'zone',
      title: 'Supply/Demand Zones',
      items: [
        {
          id: 'zone-1',
          shape: 'zone',
          fromTime: '2026-03-14T00:00:00Z',
          toTime: '2026-03-15T00:00:00Z',
          y1: 148.25,
          y2: 148.65,
          color: '#ef4444',
          opacity: 0.2,
          label: 'Supply'
        }
      ]
    }
```

```
  ]
}
```

これが Chart Renderer に渡されて初めてチャート上へ表示される。

---

14. Backend アーキテクチャ

14.1 推奨ディレクトリ構成

```
apps/api/src/modules/plugins-runtime/
  plugins-runtime.module.ts
  plugins-runtime.controller.ts
  plugins-runtime.service.ts
  runtime/
    plugin-runtime.coordinator.ts
    enabled-plugins.resolver.ts
    execution-context.builder.ts
    plugin-executor.ts
    result-normalizer.ts
    timeout-runner.ts
  dto/
    run-chart-plugins.dto.ts
    run-analysis-plugins.dto.ts
  mappers/
    runtime-response.mapper.ts
  types/
    plugin-runtime.types.ts
```

14.2 Controller API

Chart Runtime

`GET /api/v1/plugins-runtime/chart`

query:

- symbol
- timeframe
- lookbackBars
- pluginIds?（任意）

Analysis Runtime

`GET /api/v1/plugins-runtime/analysis`

query:

- symbol

- timeframe
- pluginIds?

14.3 返却形式

```
export interface ChartPluginRuntimeResponse {
  requestId: string;
  symbol: string;
  timeframe: string;
  overlays: NormalizedOverlayOutput[];
  signals: NormalizedSignalOutput[];
  indicators: NormalizedIndicatorOutput[];
  pluginStatuses: RuntimePluginStatusItem[];
}
```

---

15. Enabled Plugins Resolver

15.1 責務

- installed_plugins から `isEnabled=true` を取得
- hook / capability / pluginIds 指定で絞る
- dependency 欠落を検査
- incompatible status を除外

15.2 インターフェース

```
export interface ResolveEnabledPluginsParams {
  hook: 'chart' | 'analysis' | 'summary';
  pluginIds?: string[];
  capabilitiesAny?: string[];
}
```

15.3 chart runtime 用抽出条件

- enabled
- status = enabled
- capability に `chart.overlay` または `chart.signal` または `chart.indicator`

---

16. Execution Context Builder

16.1 取得対象

Chart 用実行時は最低限以下を集める。

- candles
- latest snapshot

- indicators snapshot
- symbol settings
- user settings

16.2 lookback 制御

プラグインごとに過剰なデータを渡さないため、

- request.lookbackBars
- runtime.maxLookbackBars

の小さい方を採用する。

16.3 例

```
const effectiveLookbackBars = Math.min(
  requestedLookbackBars,
  plugin.runtime?.maxLookbackBars ?? requestedLookbackBars
);
```

---

17. Plugin Executor

17.1 実行方式

v1 では **同一プロセス内・逐次実行** を推奨する。

理由:

- 実装コストが低い
- まずは正しい出力形式を固める方が重要
- プラグイン数が少ない前提

17.2 実行アルゴリズム

1. enabled plugin list 取得
2. module 読み込み（registry 参照）
3. hook に対応する execute 関数有無確認
4. context 作成
5. timeout runner で実行
6. result normalize
7. failures を隔離して続行

17.3 timeout

```
const timeoutMs = plugin.runtime?.defaultTimeoutMs ?? 300;
```

17.4 エラー時

- 1 plugin failure で全体停止しない
- `status='error'` と warning を返す
- runtime response には pluginStatuses を含める

---

18. Result Normalizer

18.1 役割

プラグインの返却がバラバラでも、UI で扱える標準形へ変換する。

18.2 正規化責務

- null 除外
- 不正 shape 除外
- color / opacity の安全値補正
- confidence 0..1 clamp
- duplicate overlay merge

18.3 出力マージ

複数 plugin の結果は capability 別にマージする。

```
{
  overlays: [...plugin1Overlays, ...plugin2Overlays],
  signals: [...plugin1Signals, ...plugin2Signals],
  indicators: [...plugin1Indicators, ...plugin2Indicators]
}
```

---

19. Chart 側の描画構造

19.1 Frontend 推奨構成

```
apps/web/src/
  hooks/
    useChartPluginRuntime.ts
  components/chart/plugins/
    ChartPluginOverlayLayer.tsx
    ChartPluginSignalLayer.tsx
    ChartPluginIndicatorPanel.tsx
  lib/plugins/
    chart-plugin-renderers.ts
```

19.2 Chart page データフロー

```
ChartPage
  ↓
useChartData(symbol, timeframe)
  ↓
useChartPluginRuntime(symbol, timeframe)
  ↓
ChartRenderer
  ├── price candles
  ├── overlay layer
  ├── signal layer
  └── indicator layer
```

19.3 hook 仕様

```
export function useChartPluginRuntime(params: {
 symbol: string;
 timeframe: string;
 lookbackBars: number;
})
```

返却値:

```
{
 data,
 isLoading,
 error,
 refetch,
}
```

---

20. Chart Renderer 統合要件

20.1 Overlay Layer

責務:

- zones
- lines
- marker labels

20.2 Signal Layer

責務:

- buy / sell arrow

- neutral / warning markers

20.3 Indicator Layer

責務:

- 下段 pane series
- subchart lines

20.4 重要

Plugin Runtime は **描画ロジックそのもの** を返さない。返すのは標準データだけ。 UI がそれを描画する。

これにより

- React 実装変更
- chart library 変更
- plugin 実装の互換性

を分離できる。

---

21. API 契約

21.1 packages/types に追加する schema

```
export const RunChartPluginsQuerySchema = z.object({
  symbol: z.string().min(1),
  timeframe: z.string().min(1),
  lookbackBars: z.coerce.number().int().min(50).max(5000).default(500),
  pluginIds: z.array(z.string()).optional(),
});

export const RuntimePluginStatusSchema = z.object({
  pluginId: z.string(),
  pluginSlug: z.string(),
  status: z.enum(['ok', 'error', 'skipped', 'timeout']),
  durationMs: z.number().int().nonnegative(),
  message: z.string().nullable().optional(),
});

export const ChartPluginRuntimeResponseSchema = z.object({
  requestId: z.string(),
  symbol: z.string(),
  timeframe: z.string(),
  overlays: z.array(z.any()),
  signals: z.array(z.any()),
  indicators: z.array(z.any()),
  pluginStatuses: z.array(RuntimePluginStatusSchema),
});
```

21.2 DTO 方針

ここでも必ず

- `packages/types` の Zod schema を正本
- API DTO は `createZodDto()`

に統一する。

---

22. Registry 拡張

既存の `plugins.registry.ts` が DB 状態の registry なら、Runtime 用に次を追加する。

22.1 registry が保持するもの

```
export interface RegisteredPlugin {
  manifest: PluginManifest;
  installed: InstalledPlugin;
  moduleRef: FXDEPluginModule | null;
  loadedAt?: string;
}
```

22.2 必要関数

- `getEnabledPluginsForHook(hook)`
- `getPluginModule(pluginId)`
- `refresh()`
- `assertRuntimeCompatible(pluginId)`

22.3 moduleRef について

MVP では本当に動的 import 済みでも、スタブ module map でもよい。重要なのは **Runtime 実行経路を固定すること**。

---

23. 実行結果の保存方針

v1 は DB 永続保存を必須にしない。

23.1 v1

- request ごと計算
- response を frontend に返す
- optionally in-memory cache

23.2 将来

- chart runtime cache

- plugin execution history
- plugin output snapshot table

---

24. キャッシュ戦略

24.1 キャッシュキー

```
pluginRuntime:chart:{symbol}:{timeframe}:{pluginSetHash}:{lookbackBars}
```

24.2 TTL

- M1/M5: 10〜20秒
- M15/H1: 30〜60秒
- H4/D1: 120〜300秒

24.3 v1 方針

キャッシュは optional。最初はなしでもよい。

---

25. セキュリティ

25.1 禁止

- UI から plugin code 編集
- 任意コード upload 実行
- 未検証外部 URL import

25.2 実行制限

- enabled plugin のみ実行
- timeout 必須
- try/catch 必須
- debug 情報は本番で最小化

25.3 監査

将来的には

- lastExecutedAt
- errorCount
- avgDurationMs

を更新して health 表示へ接続できる。

---

26. UX 要件

26.1 Plugin 有効時に見える変化

ユーザー観点で最も重要なのはここ。

プラグインを有効化したら、Chart page で少なくとも次のいずれかが見えるべき。

- ゾーンが描画される
- ラインが出る
- 売買サイン矢印が出る
- サマリーパネルに結果が出る
- Indicator pane が追加される

26.2 表示制御

Chart UI には plugin layer visibility toggle を入れてもよい。

例:

- All Plugins
- SupplyDemandZonesPRO
- TrendBiasAnalyzer

---

27. 実装順序

Step 1

`packages/types` に runtime schema / types を追加

Step 2

`apps/api` に `plugins-runtime` module 追加

Step 3

Enabled Plugins Resolver / Context Builder / Executor / Normalizer 実装

Step 4

`GET /api/v1/plugins-runtime/chart` 実装

Step 5

`apps/web` に `useChartPluginRuntime` 実装

Step 6

Chart page に overlay / signal layer を追加

Step 7

最低 1 つの実働 plugin（例: SupplyDemandZonesPRO）を runtime 接続

Step 8

typecheck / build / runtime 動作確認

---

28. まず動かすべき MVP

最初の実働対象は 1 つでよい。

推奨 MVP plugin

Supply Demand Zones PRO

理由:

- 出力が視覚的に分かりやすい
- ゾーン描画は overlay の代表ケース
- 有効化 → チャート変化 がすぐ確認できる

MVP 完了条件

- Plugin Manager で enable
- Chart page を開く
- `/api/v1/plugins-runtime/chart?...` が呼ばれる
- response に overlay zone が返る
- chart 上に zone が表示される

---

29. Claude 実装用の完成指示

以下を Claude にそのまま渡せるようにする。

```
# FXDE Plugin Runtime 実装タスク

目的:
Plugin Manager と Chart Engine を接続し、有効化された plugin が chart 上に overlay / signal / indicator として実際に表示される runtime を追加する。

## 必須要件
- packages/types の Zod schema を正本にする
- createZodDto() を使う
- 新規トップレベルページは増やさない
- source edit 機能は作らない
- plugins-runtime module を apps/api に追加
- GET /api/v1/plugins-runtime/chart を実装
```

```
- useChartPluginRuntime hook を apps/web に追加
- Chart page に overlay/signal layer を追加
- 最初の実働 plugin は Supply Demand Zones PRO を対象にする
- plugin 有効化後に chart へ視覚変化が出ること

## 実装順序
1. packages/types に runtime schemas / response contracts 追加
2. apps/api に plugins-runtime module 実装
3. runtime coordinator / resolver / context builder / executor / normalizer 実装
4. chart runtime API 実装
5. apps/web に runtime hook 実装
6. chart renderer integration 実装
7. Supply Demand Zones PRO を runtime 接続
8. build / typecheck / lint / 動作確認

## 完了条件
- plugin enable 後に chart に overlay または signal が出る
- runtime failure があっても全体停止しない
- pluginStatuses が返る
- typecheck 0
- build 成功
```

---

30. ChatGPT 継続監査用メモ

次の会話では以下を監査すべき。

- `packages/types` に runtime schema が追加されたか
- `createZodDto()` で runtime DTO が作られているか
- `apps/api/src/modules/plugins-runtime` が追加されたか
- enabled plugin のみ実行しているか
- chart runtime API が返す overlays/signals/indicators が標準化されているか
- Chart page が runtime response を描画に使っているか
- SupplyDemandZonesPRO 有効化時に視覚変化が出るか

---

31. 最終結論

今必要なのは Plugin Manager の追加修正ではなく、**Plugin Runtime の実働接続** である。

この Runtime を入れてはじめて、

- 有効化したプラグインが
- 実際にチャートへ作用し
- サインやゾーンが表示される

状態になる。

つまり、あなたが期待している

- 「有効化したらチャートに出る」

という仕様の本体は、まさにこの **Plugin Runtime** である。